Experiment 4: Bucket Sort

Aim

WACP to store FY % of students in an array. Write a functio for sorting array of floating-point numbers in ascending order using Bucket sort & display top five scores.

Objectives

i. Understand Bucket sort
ii. Implement Bucket Sort using functions.
iii. Sort floating-point percentages.
iv. Display top five scores
v. Analyze complexity.

Implementation

- Platform

i. 64-bit Open source linux or its derivatives.
ii. Open source C programming tools like gcc/Eclipse Editor

- Test Cases

Test Case	Input	Expected Output
TC1	One percentage	Stored successfully
TC2	Multiple percentages	Stored correctly
TC3	Random floating values	Ascending order
TC4	Duplicate values	Duplicates retained
TC5	<5 students	Display available scores
TC6	>=5 students	Display top five scores.

Sundaram®
FOR EDUCATIONAL USE

- Time Complexities

Best case : $O(n+k)$

Avg. case : $O(n+k)$

Worst case : $O(n^2)$

Conclusion:

Bucket sort efficiently sorts uniformly distributed floating-point percentages and helps identify the top five performers.

FAQs

1. What is Bucket Sort?

Ans. It is a distribution-based, divide-and-conquer sorting algorithm. It works by partitioning an array into a number of sub-arrays called buckets, sorting each bucket individually, and then concatenating the sorted buckets sequentially to produce the final sorted array.

2. Why use it for floating-point data?

Ans. Bucket-sort handles continuous floating-point numbers naturally.

i. Mathematical Index Mapping: For floating-point numbers uniformly distributed in a range, the index can be computed directly using simple arithmetic: [n x value]

ii. Linear Time Performance: It maps dense decimal values directly into discrete ranges, achieving $O(n)$ avg-time complexity w/o requiring discrete integer keys or comparison-based tree partitioning.

Sundaram®

FOR EDUCATIONAL USE

3. What is a bucket?

Ans. A bucket is a dynamic data structure designated to hold a specific sub-range of the overall input value domain.
Eg. if sorting numbers b/w 0.0 and 1.0 across 10 buckets,
Bucket 0 holds [0.0, 0.1), Bucket 1 holds [0.1, 0.2), ...

4. Worst-case complexity.

Ans. Time complexity : $O(n^2)$ - occurs when the input distribution is skewed and all n elements hash into a single bucket, which is then sorted using an $O(n^2)$ algorithm like Insertion Sort.

Space complexity: $O(n+k)$ - auxillary space to allocate k buckets and store n total elements across them.

5. Applications

Ans. i. Uniformly Distributed Floating-Point Data.
ii. External Sorting & Large Datasets.
iii. Radix Sort Subroutine
iv. Geospacial & spacial indexing.

- **Pseudocode.**

```
Structure Node:
        data: float
        next: pointer to Node

Function insertSorted (head, value):
        newNode = allocate Node
```

Sundaram®
FOR EDUCATIONAL USE

```
    newNode.data = value
    newNode.next = NULL

    If head is NULL or head.data >= value:
        newNode.next = head
        Return newNode

    current = head
    While current.next is not NULL and current.next.
    data < value:
            current = current.next

    newNode.next = current.next
    current.next = newNode
    Return head

Function inputPercentages(arr, n):
        For i from 0 to n-1:
            Input arr[i]

Function displayPercentages(arr, n):
        For i from 0 to n-1:
            Print arr[i]

Function bucketSort(arr, n):
        numBuckets = 10
        buckets = array of numBuckets Node pointers,
                initialized to NULL
```

Sundaram®
FOR EDUCATIONAL USE

```
For  i from 0 to n-1:
        bucketIndex = floor(arr[i]/10)
        If  bucketIndex >= numBuckets:
            bucketIndex = numBuckets - 1
        buckets[bucketIndex] = insertSorted(buckets[bucketIndex]
                                            , arr[i])

    idx = 0
    For i from 0 to numBuckets-1:
        current = buckets[i]
        While current is not NULL:
            arr[idx] = current.data
            idx = idx + 1
            current = current.next
```

```
Function displayTopFive(arr, n):
    count = 0
    For i form n-1 down to 0:
        Print arr[i]
        count += 1
        If count == 5:
            Break
```

```
Function Main():
    Declare arr as float array of maximum size
    Declare n=0, choice
    Declare isSorted = False

    Do:
```

Sundaram®
FOR EDUCATIONAL USE

Print "\n --- Student Percentage Management System ---"
Print " 1. Enter Student Percentages"
Print" 2. Display Current Percentages"
Print" 3. Sort Percentages (Bucket Sort)"
Print" 4. Display Top 5 Scores"
Print"5. Exit"
Input choice

Switch choice:
    Case 1: Input n
        Call input Percentages (arr, n)
        is Sorted = False
        break
    Case 2: If n == 0:
        Print "No student data available."
    Else:
        Call display Percentages (arr, n)
    Break
Case 3: If n==0: Print "No data to sort. Enterdata"
    Else: Call bucketSort (arr, n)
        is Sorted = True
        Print "Sorting Successful"
    Break.
Case 4: If n==0: Print "No data available"
    Else if is.Sorted == False:
        Print "Sort the array first"
    Else: Call display TopFive (arr, n)
    Break
Case 5: Print "Exiting Program" Break.
Default: Print "Invalid choice! Pls try again"
while choice != 5

Sundaram®
FOR EDUCATIONAL USE

Code:

```
#include <stdio.h>

#define MAX 100

// Function to perform Bucket Sort on floating-point percentages
void bucketSort(float arr[], int n) {
    float buckets[MAX][MAX];
    int count[MAX] = {0};
    int i, j, k;
    float temp;

    // 1. Scatter: Place elements into buckets based on integer part of
percentage
    for (i = 0; i < n; i++) {
        int index = (int)arr[i]; // E.g., 85.75 -> bucket index 85
        if (index >= 0 && index < MAX) {
            buckets[index][count[index]++] = arr[i];
        }
    }

    // 2. Sort: Sort each bucket individually using Insertion Sort
    for (i = 0; i < MAX; i++) {
        for (j = 1; j < count[i]; j++) {
            temp = buckets[i][j];
            k = j - 1;
            while (k >= 0 && buckets[i][k] > temp) {
                buckets[i][k + 1] = buckets[i][k];
                k--;
            }
            buckets[i][k + 1] = temp;
        }
    }

    // 3. Gather: Concatenate elements from all buckets back into the
```

```
original array
    k = 0;
    for (i = 0; i < MAX; i++) {
        for (j = 0; j < count[i]; j++) {
            arr[k++] = buckets[i][j];
        }
    }
}

int main() {
    float percentage[MAX];
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    printf("Enter student percentages:\n");
    for (i = 0; i < n; i++) {
        printf("Student %d percentage: ", i + 1);
        scanf("%f", &percentage[i]);
    }

    // Perform Bucket Sort
    bucketSort(percentage, n);

    // Display all sorted percentages
    printf("\n----------------------------------\n");
    printf("Sorted Percentages (Ascending Order):\n");
    printf("----------------------------------\n");
    for (i = 0; i < n; i++) {
        printf("%.2f ", percentage[i]);
    }
    printf("\n");

    // Display top five scores (highest scores at the end of ascending array)
    printf("\n----------------------------------\n");
```

```
    printf("Top Five Scores:\n");
    printf("-----------------------------------\n");
    int limit = (n < 5) ? n : 5; // Handle case where students count < 5
    int rank = 1;
    for (i = n - 1; i >= n - limit; i--) {
        printf("Rank %d: %.2f%%\n", rank++, percentage[i]);
    }

    return 0;
}
```

Input:

65.10
72.30
78.50
88.20
88.90
92.40
95.80

Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe
Enter number of students:7

Enter student percentages:
Student 1 percentage:78.50

Student 2 percentage:92.40

Student 3 percentage:65.10

Student 4 percentage:88.90

Student 5 percentage:95.80

Student 6 percentage:88.20

Student 7 percentage:72.30


----------------------------------------
Sorted Percentages (Ascending Order):
----------------------------------------
65.10 72.30 78.50 88.20 88.90 92.40 95.80

----------------------------------------
Top Five Scores:
----------------------------------------
```

```
----------------------------------------
Top Five Scores:
----------------------------------------
Rank 1: 95.80%
Rank 2: 92.40%
Rank 3: 88.90%
Rank 4: 88.20%
Rank 5: 78.50%

Process finished with exit code 0
```